
Playing Minecraft with LLM Agents: A New, Low-Cost Approach and Framework

Nico Adamo
nadamo@caltech.edu

Jadelynn Dao
jdao@caltech.edu

Katherine Xu
katherine@caltech.edu

Abstract

We introduce WAYFARER, an innovative LLM-powered embodied lifelong learning agent in Minecraft that excels at zero-shot generalization, automatic curriculum generation, and prompt efficiency. WAYFARER leverages an automatic curriculum driven by GPT-4o to maximize exploration and task completion using a flexible controller framework, MINEWIRE. WAYFARER employs high-level task planning and tree-like decision-making, enabling it to achieve complex crafting goals with minimal computational expense. This approach significantly reduces token usage and costs, allowing WAYFARER to craft a diamond pickaxe for mere cents. Our agent’s capabilities are further enhanced by its ability to interpret and execute tasks dynamically, adapting to new environments and challenges on-the-fly. Wayfarer demonstrates exceptional proficiency in Minecraft, achieving tech tree milestones and zero-shot crafting in consistently less iterations than existing SOTA. The full codebase, prompts, and a video demo can be found [here](#).

1 Introduction

Open-World Long-Horizon Planning Long-horizon planning poses a significant challenge in the field of machine learning, and is a bottleneck for embodied agents that continuously explore open-ended worlds [6]. However, it is difficult for traditional reinforcement learning frameworks. This complexity arises from the need to accurately predict long-term outcomes from current actions, and this is further compounded in an open-world setting, where the agent must choose from a practically infinite set of possibilities under constantly changing states/conditions. Even in RL agents that attempt this by trying to learn general purpose skills, it requires large amounts of play demonstrations as well as meticulously crafted reward functions, and in open-world settings, this becomes exponentially difficult. [5]

An ideal test bed to train long-horizon planning agents is a complex and unpredictable environment with a rich interaction space [7] [2]. In Minecraft, players have an almost unlimited set of choices in which to build skills, gather resources, etc. It also scales in complexity, from simpler tasks at the beginning of the game (breaking wood, crafting first tools, etc.), scaling to more difficult tasks later on that require extensive planning and preparation (killing the Ender Dragon, going to the Nether, etc.).

LLM-based Approaches to Minecraft Recently, LLMs have been utilized for this open-world planning — namely, a state-of-the-art agent named VOYAGER [11] uses LLM capabilities to interpret in-game events, make decisions, execute strategies, etc. Using their automatic curriculum, skill library, and iterative prompting, it is able to navigate, build, and advance through the game without any human intervention, learning along the way. Other methods make use of similar lifelong features - Ghost in the Minecraft (GITM) [14] has a memory of previous successful plans which is included in action prompts, and DEPS [12] includes a trainable module for refining its subtask selection over time. All of these methods have shown massive advantages compared to previous hierarchical RL

approaches [4] [9], due to LLM’s internet-scale knowledge of the game and zero-shot ability to reason and plan in a complex environment which traditional RL lacks.

1.1 A Need for Controller Standardization

One commonality between all LLM-agent approaches to Minecraft is the use of a low-level controller to translate commands ("craft wooden_pickaxe") into actual game inputs. This is because the LLM is making decisions based on some state information, which can only be communicated via tokens - but the actual state space of Minecraft, including details such as every block that the player can see, current location and orientation, means the spatial reasoning ability, game expertise, and associated token cost would have to be enormous to have the agent operate directly in input space. At the moment, such approaches without vision capabilities seem unlikely. Therefore, LLM-agentic methods tend to operate in a higher-level action space which controls a pathfinding algorithm or other separate tools to actually give the game specific inputs.

However, this higher-level action space is not at all standardized, calling into question the comparability of different LLM-based planning methods. DEPS [12], for example, trains an RL controller to perform a few basic tasks, but both Voyager [11] and GITM [14] have the LLM output "Mineflayer" [10] code, a pre-existing framework for writing Minecraft bots which includes pathfinding, inventory management, and other utility functions. The use of Mineflayer, as well as its notably absent presentation in these papers, obscures the line between the LLM’s capabilities and Mineflayer’s own impressive capabilities. For example, a demo video may show the agent methodically exploring a cave system in order to find diamonds over several minutes, and then making those diamonds into a diamond pickaxe - a very impressive feat by RL-based Minecraft bot standards, but somewhat diminished by the fact that all the LLM had to do to accomplish this is output the code:

```
await mineBlock(bot, "diamond_ore",3);

// Place the crafting table near the bot
const craftingTablePosition = bot.entity.position.offset(1, 0, 0);
await placeItem(bot, "crafting_table", craftingTablePosition);

// Craft a diamond pickaxe using the crafting table
await craftItem(bot, "diamond_pickaxe", 1);
```

Of course it is not necessarily an indictment of the LLM’s capabilities that it is using such high-level tools - LLMs simply reason better in high-level action spaces, and tool use is an important part of any agentic framework. However, the combination of objective in-game metrics (success rate in obtaining some item, distance explored, time survived, etc.) and a lack of controller standardization makes it essentially useless to compare between these LLM approaches for purposes other than playing Minecraft, i.e. as actual long-horizon planning frameworks.

This leads to the other main disadvantage of Mineflayer as a low-level controller, which is that as an external library, the tools it provides are not customizable. Being primarily built as a framework for automatically controlling simple bots via Javascript, it does not highly prioritize its pathfinding or other "survival" tools, and much more robust and advanced pathfinding tools exist for Minecraft such as Baritone [3], the current state-of-the-art. But being a purely server-side tool, Mineflayer has no way to interface with client-side mods like Baritone. It also has no way to support different planning settings, such as LiDAR-based sensing, or new / diminished player capabilities (i.e. no jumping, wheel-based movement, etc) which might be necessary for an accurate simulation of a planning environment.

All of these points lead us to assert the need for a standardized library which:

1. Allow comparison between different LLM-planning based methods based on the standard set of tools available to them
2. Allows for customization of these toolsets to change the action space and add or remove capabilities in order to simulate different planning environments and settings.
3. Allows for faster iteration time by being built specifically to be controlled by an LLM.

1.2 0-shot Planning in a High Level Action Space

Planning with Lifelong Agents Voyager [11] is also a "lifelong learning" agent which creates a skill library that is perpetually growing as it accumulates knowledge both from Minecraft and GPT-4. The skill library represents generalizable, executable code snippets for specific actions that the agent has learned to perform. The lifelong nature of Voyager allows it to not only reuse these skills but also apply them in new, relevant contexts as it encounters more complex challenges [8]. As Voyager continues to progress, its underlying skill library will continue to progress and build on itself.

However, lifelong learning also comes with drawbacks. Learning skills through automatic exploration is time-consuming and expensive - using GPT-4 at current prices, 60 iterations of Voyager (the minimum observed for an agent to find diamonds) costs upwards of \$50. Even outside the learning process, the tokens needed for task inference scale with the size of the learned skill library, because every single skill must be present in the prompt. These costs beg the question: is the exploratory learning and 'skill library' really worth it? Could we not simply generate code for subtasks in zero-shot manner, i.e. on the fly, rather than relying on pre-learned skills?

Voyager, of course, performs ablation which finds that the skill library is strictly necessary for more complex tasks like obtaining diamonds tools. But over the course of our testing, we noticed that this was *not* because the skill library allowed it to build up increasingly complex behaviors.

Context Failure Instead, the skill library is used by the LLM to circumvent a deficiency we will refer to as "context failure". Because the Mineflayer API is not built with LLMs in mind, its behaviors are relatively complex: they may involve some pathfinding, which can fail, or not - they may automatically interact with the block required for the action, or not. For example, the LLM might successfully obtain the materials to craft an iron pickaxe and then output the code `craftItem(bot, "iron_pickaxe", 1)`, which then fails for any number of reasons:

1. It hasn't placed a crafting table, but has one in its inventory
2. It has placed a crafting table but is 6 blocks from it
3. It has placed a crafting table but is 1000 blocks from it
4. It has the materials to craft a crafting table but has not yet
5. It does not have the materials to craft a crafting table
6. ...

All of these failure modes must be dealt with in different ways. The massive amount of information which Voyager prompts the LLM with and its iterative retrying allows it to distinguish between these failure cases and refine its "crafting skill" to slowly account for each of these failure cases, rather than dealing with them over again each time. In this way, the primary use of the skill library is to account for unintuitive behaviors of the Mineflayer API.

High-Level Action Spaces Some level of unintuitive behavior is implicit in any reasonable controller, given that the state space of Minecraft is so large and varied, and depending on player state, tasks like "crafting an item" can sometimes translate to a few mouse clicks, or a very complicated series of context-dependent actions. But we postulate that the Mineflayer API is far from optimal in this regard. Previous research [1] [12] suggests that hierarchical approaches which connect the reasoning space of an LLM (natural language) to its action space (controller commands) let the LLM avoid making "logical jumps" and improve plan robustness and the semantic relevance of chosen actions. With this in mind, we propose that an action space specifically crafted for an LLM could dramatically lower the required reasoning ability for an LLM to understand a command's failure modes and how to recover based on the current player state. Specifically, tools which are:

1. More explicit in their scope and failure modes
2. Have better built-in failure recovery
3. Provide a higher-level interface for the LLM to use, such that housekeeping tasks like inventory management are abstracted away entirely

These elements together allow the LLM to reason in a very similar space to its action space, bridging the gap between planning and acting. Within this framework, we propose that an expensive skill

library will not be necessary, token costs will be lower on the whole, and plans will be more robust and recoverable.

2 MineWire

We introduce **MineWire**, a flexible and modular controller framework built as a client-side mod for Minecraft and packaged Java / Python API which allows for fine-grained control and customization of an LLM agent’s action space and a powerful library of built-in tools.

The core of MineWire is the task abstraction. In MineWire, the space of tasks is the action space of the LLM - a plan consists of an ordered list of tasks. The most fundamental type of task is a Basic task, which contains `execute`, `getState`, and `cleanup` methods. These tasks dictate exactly which game inputs are pressed on every frame based only on client-side logic - i.e. `GotoBlock`, which uses Baritone pathfinding to determine the quickest path to the nearest block of a certain type and follows it. `getState` is called periodically to check the status of execution - once a block of the requested type is nearby, for example, `getState` will return a terminated success state. If the pathfinding is unsuccessful, `getState` will return a terminated failure state with some failure reason attached.

The second type of task is a composite task, which can be made up of one or more Basic tasks based on client-side logic. For example, the "craft" task looks up the recipe for the requested item, and if it is a 2x2 recipe, is made up of a `OpenInventory` task followed by a `BasicCraft` task - otherwise, it is made up of a `FindOrPlaceBlock("crafting_table")`, `InteractBlock("crafting_table")` and finally a `BasicCraft` task. Composite tasks take the place of a basic skill library, allowing the LLM to focus on high-level planning rather than rederiving common patterns to satisfy the arbitrary behaviors and requirements of a vague controller API.

Tasks can be exposed as commands usable by the LLM, or not. In our testing we found it was helpful to obscure some lower-level commands, such as `OpenInventory`, from the knowledge of the LLM, as they were only used as part of Compound Tasks and could confuse the agent by giving them multiple ways to accomplish the same task (by expanding the compound task’s basic components itself). Fig 1 shows the set of commands which we exposed in our experiments. The MineWire client mod exposes a convenient API to register new tasks and commands as long as they conform to the expected task interface.

The MineWire client mod communicates with Python via a websocket message bus run as a local server. The client acts as a sentinel to receive messages and send important information back to the Python controller, which actually controls the agent. The types and content of messages are highly customizable, but by default, messages are sent for the following reasons:

1. The user sets a goal for the agent.
2. A task execution fails.
3. A plan succeeds.
4. The LLM requests to modify the existing plan or re-plan from the current point.
5. The LLM generates an initial plan.

This event abstraction takes care of keeping track of game state and plan status in real-time, but leaves the actual planning method, implemented on the Python side, completely open to implementation. Global state, such as memory or skill libraries, can easily be kept. Task feedback and current agent state can be ignored, or used to re-plan. All of this serves to connect the LLM as closely to the in-game agent as possible, so it can focus on its strength of high-level planning, rather than trying to tame a complex controller interface or deal with out-of-date or irrelevant data.

3 0-Shot Planning with MineWire

Armed with a higher-level action space and more salient, precise tools for our agent, we can now ask the question: can 0-shot planning with feedback, but no global state or memory, be a low-cost, effective long-horizon strategy?

Our planning system consists of two main components. The LLM Planning module breaks down a user-provided goal into a list of subgoals, using pure natural language reasoning and chain-of-thought

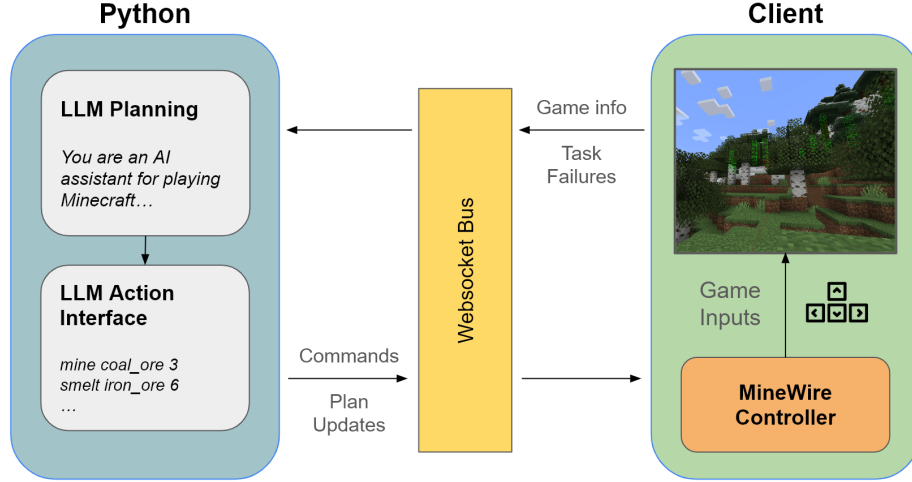


Figure 1: MineWire Architecture. The websocket messaging system allows flexibility in when and how planning and re-planning occur, and the task system gives the LLM a clear, salient action space while allowing for high customization of the planning setting and fast iteration time for building competent agents.

Table 1: Default MineWire Commands

MineWire Command	Description	Example
goto_block	Go to a specific block type.	goto_block ender_chest
mine	Mine a block until a specified amount is in the inventory.	mine 64 diamond_ore
craft	Craft an item using inventory or crafting table.	craft 2 stick
smelt	Smelt an item using a furnace.	smelt 2 iron_ingot
place	Place a block in the world.	place crafting_table
goto_y_level	Go to a specific y-level in the world.	goto_y_level 12
wander n	Travel n blocks in any direction	wander 100

prompting [13]. No constraints are put on the output of the module, except that it produces a list of steps to take, bringing it closer to the agent’s eventual action space. This module additionally receives some state information from the client including the agent’s current inventory and biome.

Next, the Action Interface translates this list of subgoals into a list of tasks, using the provided command interface. This plan is passed to the client via the websocket, which begins execution of each task. When a task fails, the failure reason is passed back to the planning module in the Python controller, along with the current player state information. Following CoT [13] techniques, we have the LLM synthesize the failure reason and state information into a plausible natural-language explanation of why the task failed and the recovery plan. This new plan is then translated into a list of tasks using the action interface, and the client updates the agent’s trajectory accordingly.

Auto-Goaling To directly compare with Voyager as a lifelong agent, we also include an autogoal module which automatically builds an exploration curriculum as the agent progresses through the game. Upon starting the Wayfarer agent, it generates a starter task that is relevant considering the state of the game. Once the goal is determined, we query the LLM as described above for gameplay steps. Upon the completion of each generated goal, it prompts the LLM to consider the current game progress and state, the player’s inventory, and any previously completed goals to generate a new appropriate goal ensuring a continuous and logical progression through the game. This means that with auto-goaling, Wayfarer can be fully autonomous, continuously generating and pursuing new, meaningful goals.

4 Experiments

4.1 Experimental Setup

To evaluate the performance of our proposed LLM agent in Minecraft, we conducted a series of experiments designed to test both skill mastery and zero-shot generalization capabilities. We used the same Minecraft environment settings as described in the VOYAGER paper to ensure a fair comparison. The agent’s state was reset at the beginning of each experiment, clearing its inventory and environmental changes. The agent was then tasked with progressing through the Minecraft tech tree and completing specific unseen tasks. We measured both the number of iterations and the number of tokens consumed during each task as metrics of efficiency, given their significant impact on computational cost.

4.2 Previous State-of-the-Art (SOTA)

We compare our agent’s results against empirical results referenced in the VOYAGER paper. Specifically, we compare our method to:

- **Voyager**: The baseline method which uses a lifelong learning approach with a skill library.
- **Voyager without skill library**: A variant of the baseline which does not utilize the skill library, thus relying solely on the LLM’s immediate problem-solving capabilities without leveraging past learned skills.

Using Voyager for approximately 160 iterations using GPT-4 will cost around 50 USD, meaning that each iteration costs approximately \$0.31.

4.3 Results

We chose to focus on the metrics of tech tree mastery and zero-shot generalization to unseen tasks primarily because these were the metrics used by Voyager in their evaluations. We focused on evaluation metrics from Voyager that could be translated to this zero-shot approach. This allows for a direct comparison and helps establish the effectiveness of our proposed method in a well-defined context.

Tech Tree Mastery

The Minecraft tech tree tests the agent’s ability to craft and use a hierarchy of tools, progressing from wooden tools to stone, iron, and finally diamond tools. This progression requires the agent to master systematic and compositional skills through an automatic curriculum. The curriculum generates tasks dynamically based on the agent’s current state and past progress.

To evaluate efficiency in tech tree mastery, we compared both the number of iterations and the number of tokens required for the agent to complete each stage of the tech tree. Token usage is particularly indicative of the computational cost, as it directly impacts the overall expense of running the LLM.

Table 2: Comparison of Iterations and Token Usage for Tech Tree Mastery Using Auto-Goaling

Agent	Wooden Tools		Stone Tools		Iron Tools		Diamond Tools	
	Iterations	Tokens	Iterations	Tokens	Iterations	Tokens	Iterations	Tokens
Voyager w/o Skill Library	7 $\pm$ 2	104,000	9 $\pm$ 4	180,000	29 $\pm$ 11	712,000	N/A	N/A
Voyager	6 $\pm$ 2	101,000	11 $\pm$ 2	230,000	21 $\pm$ 7	452,000	102	2,421,000
Wayfarer (ours)	1 $\pm$ 0	1500	1 $\pm$ 1	2000	4 $\pm$ 2	6000	7 $\pm$ 4	10500

Zero-shot Generalization to Unseen Tasks

To evaluate zero-shot generalization, we cleared the agent’s inventory, reset it to a newly instantiated world, and tested it with unseen tasks such as crafting a Diamond Pickaxe and Golden Sword.

Similar to the tech tree mastery evaluation, we measured both the number of iterations and the number of tokens consumed to complete each task. These metrics provide a clearer picture of the computational efficiency and potential cost savings of our proposed method.

Table 3: Comparison of Iterations and Token Usage for Zero-shot Generalization

Agent	Diamond Pickaxe		Golden Sword	
	Iterations	Tokens	Iterations	Tokens
Voyager w/o Skill Library	36	715,000	30 ± 9	630,000
Voyager	19 ± 3	402,000	18 ± 7	360,000
Wayfarer (ours)	7 ± 4	14,000	7 ± 2	14,000

Our results demonstrate that the proposed method is more efficient in terms of both iterations and token usage compared to the current SOTA methods. The reduction in tokens required for both tech tree mastery and zero-shot generalization tasks indicates a significant improvement in computational efficiency, which translates to cost savings when using LLMs for complex, open-world planning tasks in environments like Minecraft.

5 Discussion

Our results demonstrate that our proposed Wayfarer agent is more efficient in terms of both iterations and token usage compared to the current state-of-the-art methods. The significant reduction in tokens required for both tech tree mastery and zero-shot generalization tasks indicates a substantial improvement in computational efficiency, translating directly to cost savings when using LLMs for complex, open-world planning tasks in environments like Minecraft.

The next step in our research would be to recreate more extensive tests to validate and extend our findings. Due to time constraints and the scope of this project, we did not consider other crafting metrics that were evaluated in Voyager, such as crafting a compass or other complex items, although including such metrics should only be a question of adding Tasks allowing for the agent to kill mobs, farming, etc. Expanding our evaluation to include these additional tasks would provide a more comprehensive comparison and further demonstrate the robustness of our approach.

5.1 Limitations

Qualitative observations indicate that some failures occurred when the agent attempted to craft items with materials already in its inventory but were not recognized. For example, the agent often failed to recognize planks required to build sticks, despite having them in its inventory. This issue likely stems from the Baritone framework our implementation rather than the GPT agent’s poor/wrong instructions. Addressing this recognition problem could further enhance performance, and future research should focus on resolving these errors.

Furthermore, our evaluation primarily concentrated on crafting capabilities, which best demonstrate the agent’s ability to build upon previous tasks. However, other metrics such as the construction of 3D structures (e.g. a house) or the combating of hostile entities such as creepers were not considered in this study. Including these additional metrics in future evaluations could provide a more comprehensive assessment of the agent’s overall game mastery.

6 Conclusion

We presented a novel LLM-based approach for long-horizon open-world planning in Minecraft. Our method achieves significant improvements in zero-shot accuracy and computational efficiency compared to the current state-of-the-art method. Our approach demonstrated superior performance in tech tree mastery and zero-shot generalization tasks, achieving complex goals such as crafting a diamond pickaxe at a fraction of the prompting lengths previously required. While our method showed promising results, we identified limitations related to common iteration failures - future work should address these issues to further enhance the agent’s capabilities and provide a more holistic evaluation of its performance in complex, open-world environments.

References

- [1] J. He, S. Chen, F. Zhang, and Z. Yang. From words to actions: Unveiling the theoretical underpinnings of llm-driven autonomous systems, 2024.
- [2] I. Kanitscheider, J. Huizinga, D. Farhi, W. H. Guss, B. Houghton, R. Sampedro, P. Zhokhov, B. Baker, A. Ecoffet, J. Tang, O. Klimov, and J. Clune. Multi-task curriculum learning in a complex, visual, hard-exploration domain: Minecraft, 2021.
- [3] Leijurv. Baritone: Pathfinding for minecraft, 2019. URL <https://github.com/cabaletta/baritone>.
- [4] Z. Lin, J. Li, J. Shi, D. Ye, Q. Fu, and W. Yang. Juewu-mc: Playing minecraft with sample-efficient hierarchical reinforcement learning, 2021.
- [5] C. Lynch, M. Khansari, T. Xiao, V. Kumar, J. Tompson, S. Levine, and P. Sermanet. Learning latent plans from play, 2019.
- [6] M. Savva, A. Kadian, O. Maksymets, Y. Zhao, E. Wijmans, B. Jain, J. Straub, J. Liu, V. Koltun, J. Malik, D. Parikh, and D. Batra. Habitat: A platform for embodied ai research, 2019.
- [7] B. Shen, F. Xia, C. Li, R. Martín-Martín, L. Fan, G. Wang, C. Pérez-D’Arpino, S. Buch, S. Srivastava, L. P. Tchapmi, M. E. Tchapmi, K. Vainio, J. Wong, L. Fei-Fei, and S. Savarese. igibson 1.0: a simulation environment for interactive tasks in large realistic scenes, 2021.
- [8] D. I. A. Team, J. Abramson, A. Ahuja, A. Brussee, F. Carnevale, M. Cassin, F. Fischer, P. Georgiev, A. Goldin, M. Gupta, T. Harley, F. Hill, P. C. Humphreys, A. Hung, J. Landon, T. Lillicrap, H. Merzic, A. Muldal, A. Santoro, G. Scully, T. von Glehn, G. Wayne, N. Wong, C. Yan, and R. Zhu. Creating multimodal interactive agents with imitation and self-supervised learning, 2022.
- [9] C. Tessler, S. Givony, T. Zahavy, D. J. Mankowitz, and S. Mannor. A deep hierarchical approach to lifelong learning in minecraft, 2016.
- [10] A. Unknown. Mineflayer: A minecraft bot library, 2024. URL <https://github.com/PrismarineJS/mineflayer>. Accessed: 2024-06-04.
- [11] G. Wang, Y. Xie, Y. Jiang, A. Mandlkar, C. Xiao, Y. Zhu, L. Fan, and A. Anandkumar. Voyager: An open-ended embodied agent with large language models, 2023.
- [12] Z. Wang, S. Cai, G. Chen, A. Liu, X. Ma, and Y. Liang. Describe, explain, plan and select: Interactive planning with large language models enables open-world multi-task agents, 2023.
- [13] J. Wei, X. Wang, D. Schuurmans, M. Bosma, B. Ichter, F. Xia, E. Chi, Q. Le, and D. Zhou. Chain-of-thought prompting elicits reasoning in large language models, 2023.
- [14] X. Zhu, Y. Chen, H. Tian, C. Tao, W. Su, C. Yang, G. Huang, B. Li, L. Lu, X. Wang, Y. Qiao, Z. Zhang, and J. Dai. Ghost in the minecraft: Generally capable agents for open-world environments via large language models with text-based knowledge and memory, 2023.